



Programación Gestión de bases de datos

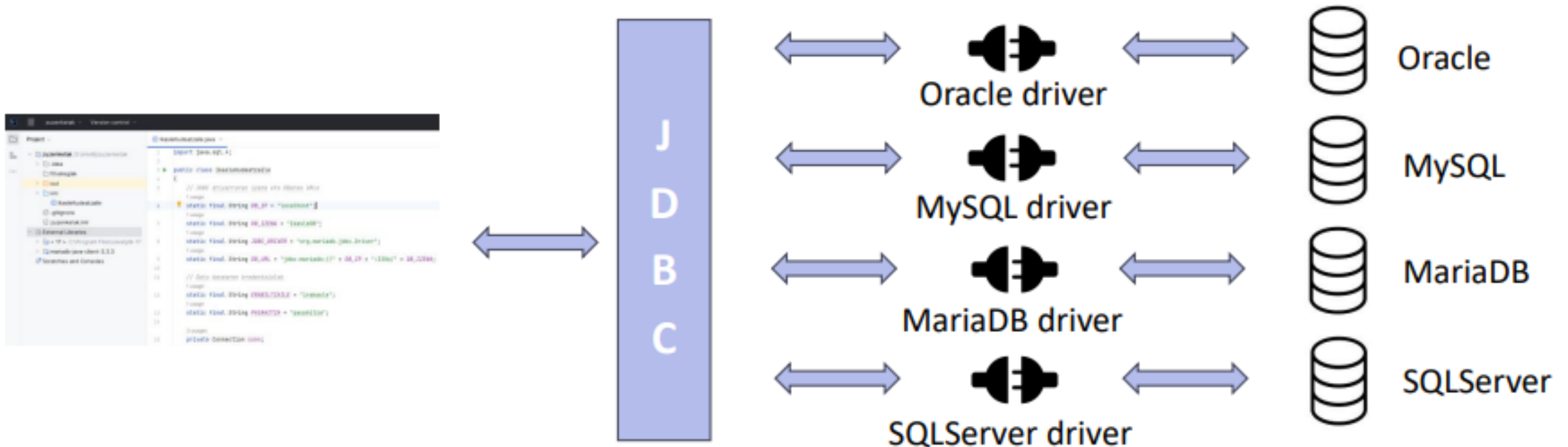


JDBC (Java Database Connectivity)

- Es una interfaz de Java para trabajar con bases de datos relacionales
 - - Es independiente del sistema
 - - Es transparente para la base de datos utilizada, es decir, utiliza los mismos métodos para trabajar con cualquier base de datos relacional (Oracle, MySQL, SQL Server, Access...).
- Para conectarse a una base de datos, una aplicación necesita tres elementos:
 - - Kit de Desarrollo de Java (JDK)
 - - Biblioteca JDBC (incluida en el JDK)
 - - Controlador JDBC para la base de datos que se va a utilizar

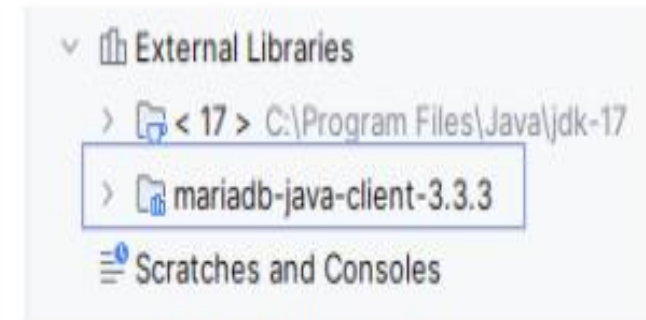
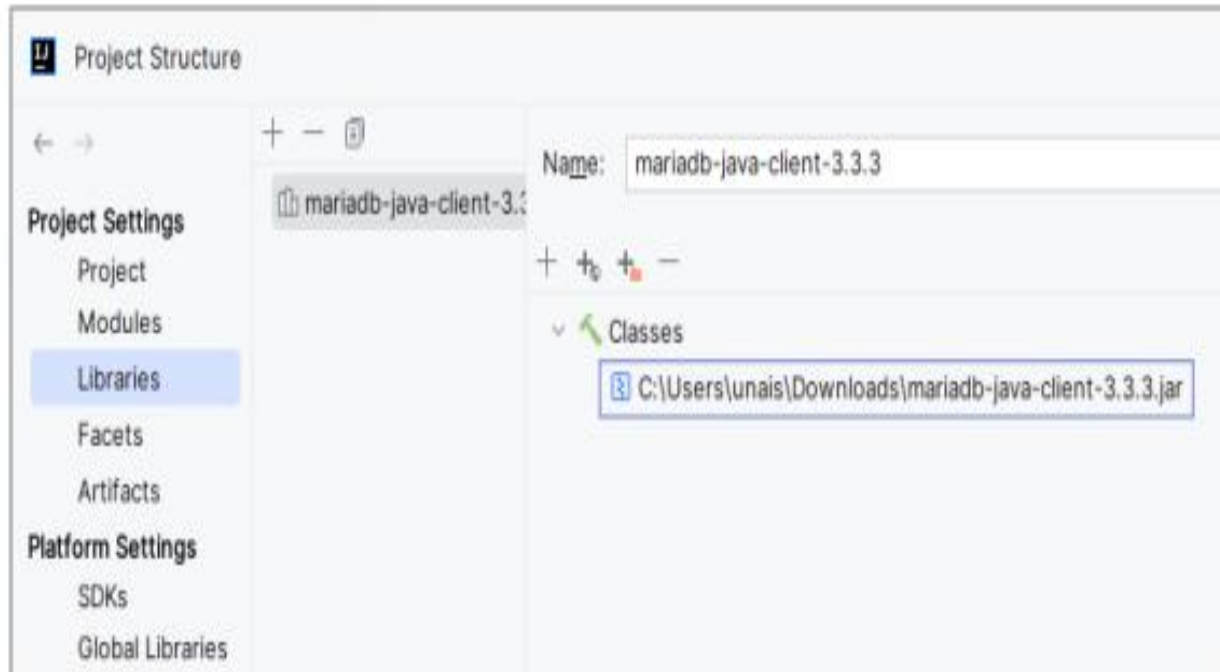
Drivers en JDBC

- Para usar JDBC con una base de datos específica, se requiere un driver que actúe como puente entre la base de datos y la biblioteca JDBC.
 - Implementa los métodos de la interfaz JDBCDriver.
 - Traduce las llamadas de la biblioteca JDBC al lenguaje de la base de datos y viceversa.



Drivers en JDBC

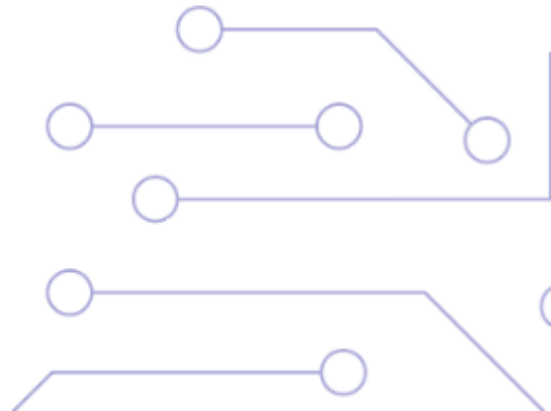
- Vienen en un archivo JAR que debe agregarse al proyecto como una biblioteca externa.



La biblioteca JDBC

- Es responsable de tres tareas principales:
 - Gestionar la conexión a la base de datos (abrir/cerrar)
 - Crear y ejecutar consultas SQL
 - Gestionar resultados

```
Connection conn = DriverManager.getConnection("jdbc:odbc:xxxxxx",
"usuario ", "clave");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT a, b, c FROM Tabla1");
while (rs.next())
{
    int x = rs.getInt("a");
    String s = rs.getString("b");
    float f = rs.getFloat("c");
}
```



La biblioteca JDBC

- Las clases más utilizadas están en el paquete `java.sql`

Tipo	Clase
Implementación	<code>java.sql.Driver</code> <code>java.sql.DriverManager</code> <code>java.sql.DriverPropertyInfo</code>
Conexión a Base de Datos	<code>java.sql.Connection</code>
Gestión de las consultas SQL	<code>java.sql.Statement</code> <code>java.sql.PreparedStatement</code> <code>java.sql.CallableStatement</code>
Gestión de resultados	<code>java.sql.ResultSet</code>
Gestión de errores	<code>java.sql.SQLException</code> <code>java.sql.SQLWarning</code>

La clase SQLException

- La mayoría de los métodos JDBC lanzan esta excepción.
 - Se lanza al conectarse a la base de datos, al realizar una petición, cuando no hay suficientes permisos, etc.
 - Es necesario gestionar la excepción dentro de un bloque *try-catch*.

```
try
{
    ...
}
catch (SQLException e)
{
    e.printStackTrace();
}
```

La clase DriverManager

- Esta clase se encarga de registrar controladores de base de datos específicos y de conectarse a la base de datos.
- En lugar de usar el método `registerDriver` para registrar el controlador, se utiliza el método `Class.forName(nombre del controlador)`.
 - Este método llama al método `registerDriver`

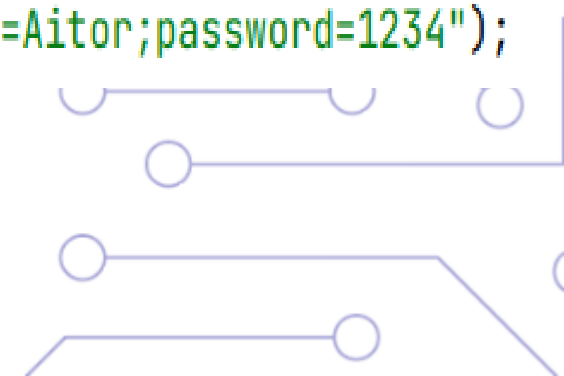
```
Class.forName("org.mariadb.jdbc.Driver" );  
Class.forName("com.mysql.cj.jdbc.Driver" );  
Class.forName("org.postgresql.Driver");  
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
Class.forName("oracle.jdbc.driver.OracleDriver");  
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
```

La clase DriverManager

- El método `getConnection` se utiliza para establecer una conexión a la base de datos.
 - Se le debe pasar la URL de la base de datos y, si es posible, el nombre de usuario y la contraseña.
 - Ejemplos de URL
 - `jdbc:mysql://<IPServidor>:<puerto>/<nombre BD>`
 - `jdbc:mariadb://<IPServidor>:<puerto>/<nombre BD>`
 - `jdbc:postgresql://<IPServidor>:<puerto>/<nombre BD>`
 - `jdbc.odbc:<nombreBD>`
 - `jdbc:oracle:thin:@<IPServidor>:<puerto>:<nombre BD>`
 - `jdbc:sqlserver://<IPServidor>:<puerto>;databaseName=xxx;user=xxx; password=xxx`

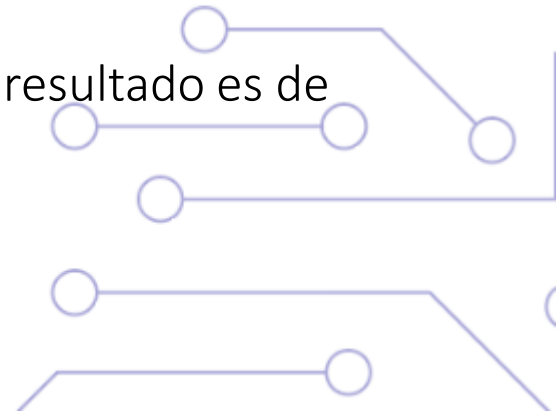
La clase DriverManager

```
Connection conn = DriverManager.getConnection("jdbc:odbc:ARTIKULUAK");  
Connection conn = DriverManager.getConnection("jdbc:oracle:thin:@host:1521:ARTIKULUAK","Aitor", "1234");  
Connection conn = DriverManager.getConnection("jdbc:mariadb://localhost:3306/dbbizikletak","Aitor","1234");  
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/dbbizikletak","Aitor","1234");  
Connection conn =  
DriverManager.getConnection("jdbc:sqlserver://localhost:1533;databaseName=Zinea;user=Aitor;password=1234");
```



La clase Connection

- Representa una conexión a la base de datos
- Su propósito principal es crear consultas SQL
 - ***Statement createStatement(int tipo, int concurrencia)***: Crea un objeto para enviar una consulta SQL a la base de datos. Tiene dos parámetros, tipo y concurrencia, que toman valores de algunos enumeradores:
 - ***ResultSet.TYPE_FORWARD_ONLY***: Valor predeterminado para el tipo. Los resultados solo se pueden analizar hacia adelante.
 - ***ResultSet.TYPE_SCROLL_INSENSITIVE***: Los resultados se pueden analizar hacia adelante y hacia atrás. No detecta cambios realizados después del resultado.
 - ***ResultSet.TYPE_SCROLL_SENSITIVE***: Los resultados se pueden analizar hacia adelante y hacia atrás. Detecta cambios realizados después del resultado.
 - ***ResultSet.CONCUR_READ_ONLY***: Valor predeterminado para la concurrencia. El resultado es de tipo lectura.
 - ***ResultSet.CONCUR_UPDATABLE***: El resultado es de tipo lectura-escritura.



La clase Connection

PreparedStatement prepareStatement(String sql):

- Toma una cadena de caracteres que contiene una consulta SQL como parámetro y crea un objeto especial de tipo *PreparedStatement*.
- Es muy útil si la misma consulta debe ejecutarse varias veces.

- CallableStatement prepareCall(String sql):

- Toma una cadena que contiene una consulta SQL como parámetro.
- Crea un objeto especial de tipo *CallableStatement*.
- Esta es una llamada a un procedimiento almacenado en una base de datos.

```
Statement consulta1 = conn.createStatement();
```

```
Statement consulta2 = conn.createStatement();
```



Se puede crear más
de una consulta en
una conexión.

La clase Connection

Otros métodos

Métodos	Definición
<code>void close()</code>	Libera la base de datos del objeto de conexión y los recursos JDBC inmediatamente (en lugar de esperar a que se liberen automáticamente).
<code>void commit()</code>	Hace que todos los cambios realizados desde el estado de commit/rollback anterior sean permanentes.
<code>void rollback()</code>	Deshace todos los cambios realizados desde el estado de commit/rollback anterior .



La clase Statement

- Se utiliza para enviar consultas SQL a una base de datos mediante una conexión y recibir los resultados. Admite consultas DQL, DML y DDL.
 - DQL (Data Query Language): Select consultas.
 - ML (Data Manipulation Language): Consultas de tipo Insert, Update, Delete.
 - DDL (Data Definition Language): Consultas de tipo Create, Drop, Alter.

La clase Statement

- Métodos:

Método	Definición
ResultSet executeQuery(String sql)	Ejecuta la consulta SQL que se le pasa y devuelve un objeto de tipo ResultSet, un conjunto de filas. Se utiliza para consultas Select.
int executeUpdate(String sql)	Ejecuta la consulta SQL que se le pasa, que puede ser de tipo DML o DDL. Las consultas DML devuelven el número de filas modificadas. Las consultas DDL devuelven 0.
boolean execute(String sql)	Ejecuta la consulta SQL que se le pasa. Se utiliza en tiempo de compilación para consultas desconocidas o que devuelven resultados complejos.
void close()	Libera la base de datos del objeto de conexión y los recursos JDBC inmediatamente (en lugar de esperar a que se liberen automáticamente)

La clase Statement

```
ResultSet productos = st.executeQuery("SELECT * FROM productos WHERE precio<10;");  
st.executeUpdate("INSERT INTO cafes VALUES ('Colombiano',9);");  
st.executeUpdate("DROP TABLE IF EXISTS personas;");  
st.executeUpdate("CREATE TABLE personas (nombre, trabajo);");
```

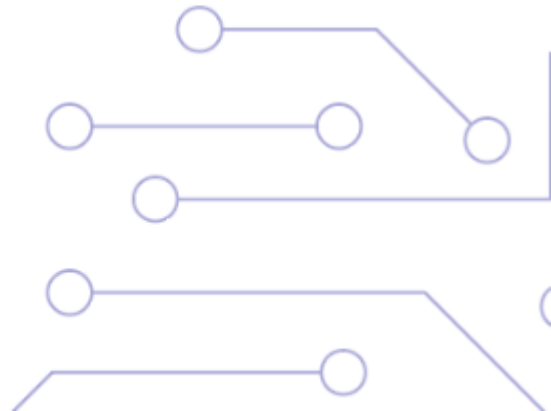
La clase ResultSet

- Almacena los resultados de una consulta SQL ejecutada.
 - Puede considerarse como una tabla con las filas y columnas devueltas.
- Navega por los resultados mediante un cursor y proporciona métodos para analizarlos:

Método	Definición
boolean next()	Mueve el cursor a la siguiente fila. Inicialmente, el cursor está antes de la primera fila, por lo que se debe realizar una llamada antes de leer la primera fila. Devuelve verdadero si la fila actual es válida y falso si no quedan filas.

La clase ResultSet

Método	Definición
<code>int getInt (int columna)</code> <code>int getInt (String columna_nombre)</code>	Devuelven el valor correspondiente a una columna de la fila actual.
<code>float getFloat (int columna)</code> <code>float getFloat (String columna_nombre)</code>	La columna se puede especificar por nombre o por posición (NOTA: ¡la posición comienza en 1!).
<code>String getString (int column)</code> <code>String getString (String columna_nombre)</code>	Los métodos <code>getString</code> permiten leer cualquier valor como una cadena de caracteres, incluidos valores numéricos.



La clase ResultSet

- Relación entre los tipos de datos SQL y los métodos getXXX

Métodos getXXX	Tipo de datos
getInt	INTEGER
getLong	BIG INT
getFloat	REAL
getDouble	FLOAT
getBignum	DECIMAL
getBoolean	BIT

Métodos getXXX	Tipo de datos
getString	VARCHAR
getString	CHAR
getDate	DATE
getTime	TIME
getTimestamp	TIME STAMP
getObject	De cualquier tipo

La clase ResultSet

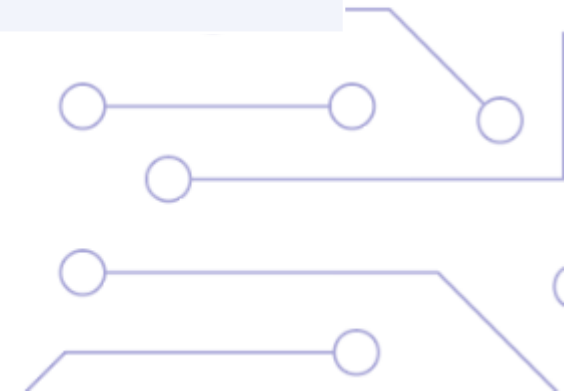
- Otros métodos:

Método	Definición
ResultSetMetaData getMetaData()	Devuelve las propiedades del resultado: número de columnas, tipo y nombre. Estos valores se pueden obtener mediante los métodos getColumnCount, getColumnType y getColumnName.
void close()	Libera la base de datos del objeto de conexión y los recursos JDBC inmediatamente (en lugar de esperar a que se liberen automáticamente).
boolean previous()	Mueve el cursor a la fila anterior al resultado.
boolean first()	Mueve el cursor a la primera fila del resultado.
boolean last()	Mueve el cursor a la última fila del resultado.
boolean absolute(int)	Mueve el cursor al número de fila pasado como parámetro.
boolean relative(int)	Mueve el cursor al número de filas pasadas como parámetro (hacia adelante o hacia atrás si el número es positivo o negativo).

La clase ResultSet

- Otros métodos:

Método	Definición
<code>void beforeFirst()</code>	Posiciona el cursor antes de la primera fila del resultado.
<code>void afterLast()</code>	Posiciona el cursor después de la última fila del resultado.
<code>boolean isFirst()</code> <code>boolean isLast()</code>	Devuelve si el cursor está en la primera/última fila del resultado.
<code>boolean isBeforeFirst()</code> <code>boolean isAfterLast()</code>	Devuelve si el cursor está antes de la primera fila o después de la última fila del resultado.



La clase ResultSet

- Otros métodos:

Método	Definición
<code>void updateXXX(zutabe, balioa)</code> <code>void updateInt(...,)</code> <code>void updateString (... ,)</code>	Actualiza la columna especificada con el valor. La columna se puede especificar por nombre o posición. <code>rs.updateFloat("precio", 13.49);</code> ATENCIÓN: Este método no modifica la base de datos subyacente.
<code>void updateRow()</code>	Actualiza la base de datos con el valor de la fila actual. <code>rs.updateFloat("precio", 13.49);</code> <code>rs.updateRow();</code>
<code>void cancelRowUpdates()</code>	Cancela los cambios realizados en la columna de la fila actual. <code>rs.updateFloat("precio", 13.49);</code> <code>rs.cancelRowUpdates();</code>

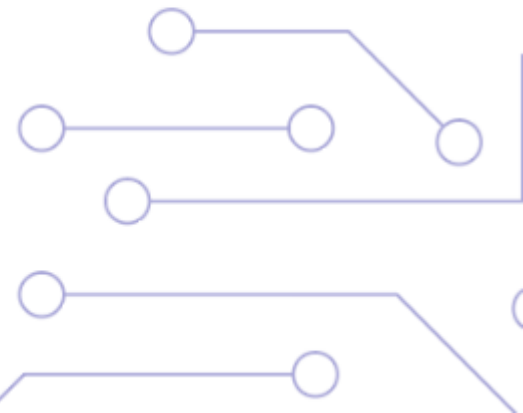
La clase ResultSet

- Otros métodos:

Método	Definición
<code>void deleteRow()</code>	Elimina la fila actual del resultado y de la base de datos.
<code>void moveToInsertRow()</code>	Se prepara para insertar una nueva fila de resultados.
<code>void insertRow()</code>	Inserta una nueva fila en el objeto de resultado y en la base de datos. <pre>rs.moveToInsertRow(); rs.updateString("Nombre", "Andoni"); rs.updateString("Apellido", "Mitxelena"); rs.updateInt("Edad", 25); rs.insertRow();</pre>

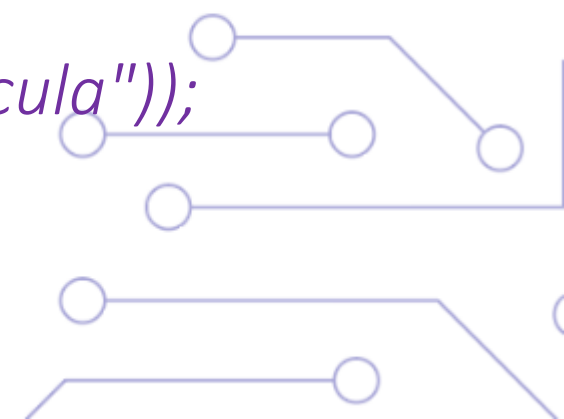
La clase ResultSet

```
String sql = "SELECT titulares.* FROM titulares WHERE DNI = '22222222B';  
Statement consulta = conn.createStatement();  
ResultSet resultado = consulta.executeQuery(sql);  
resultado.next();  
System.out.println(resultado.getString(«DNI") + " | " +  
resultado.getString("Nombre") + " | " + resultado.getInt("Edad"));
```



La clase ResultSet

```
String sql = "SELECT Matricula FROM coches WHERE Precio > 22000";  
Statement consulta = conn.createStatement();  
ResultSet resultado = consulta.executeQuery(sql);  
while(resultado.next())  
{  
    System.out.println("Matrícula:" + resultado.getString(1));  
    //System.out.println("Matrícula:" + result.getString("Matrícula"));  
}
```



La clase ResultSet

```
String sql="SELECT Propietarios.* FROM Propietarios INNER JOIN Autos ON Propietarios.DNI = Autos.DNI WHERE Precio > 22000";
```

```
Instrucción st = conn.createStatement();
```

```
ResultSet result = st.executeQuery(sql);
```

```
ResultSetMetaData md = result.getMetaData();
```

```
while(result.next())
```

```
{
```

```
    System.out.print("Propietario: | ");
```

```
    for(int i = 1; i <= md.getColumnCount(); i++)
```

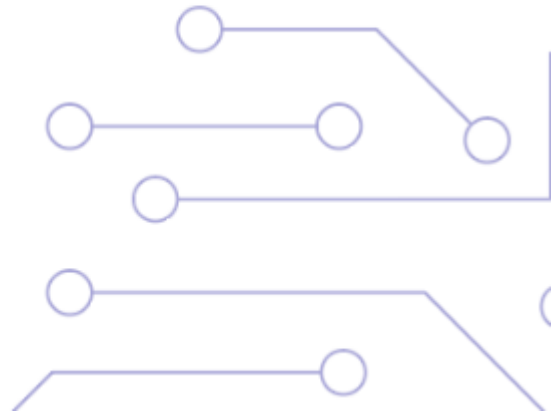
```
    {
```

```
        System.out.print(result.getString(i) + " | ");
```

```
    }
```

```
    System.out.print("\n");
```

```
}
```



La clase ResultSet

```
String sql = "ALTER TABLE Propietarios ADD VIP BIT";
```

```
Statement consulta1 = conn.createStatement();
```

```
int a = consulta1.executeUpdate(sql);
```

```
sql = "UPDATE Propietarios SET VIP = true WHERE DNI IN (SELECT DNI FROM Autos  
WHERE Precio > 22000)";
```

```
Statement consulta2 = conn.createStatement();
```

```
int b = consulta2.executeUpdate(sql);
```

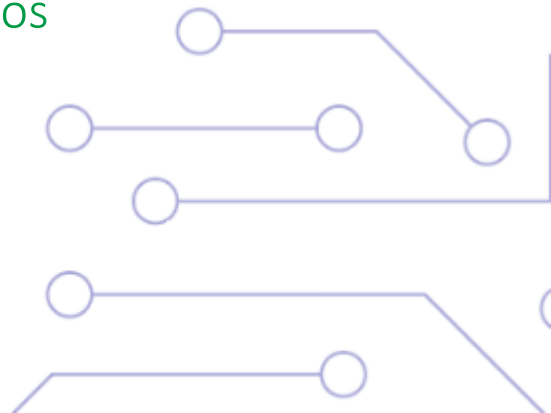
```
System.out.println(b + " propietarios han sido actualizados.");
```

```
sql = "UPDATE Propietarios SET VIP = false WHERE DNI NOT IN (SELECT DNI FROM Autos  
WHERE Precio <= 22000)";
```

```
Sección consulta3 = conn.createStatement();
```

```
int c = consulta3.executeUpdate(sql);
```

```
System.out.println(c + " propietarios han sido actualizados.");
```



La clase PreparedStatement

- Se utiliza para crear y ejecutar consultas SQL precompiladas.
 - Al estar precompilada, es ideal cuando es necesario repetir la misma consulta varias veces, ya que se ejecuta más rápido.
- En las consultas SQL, se utiliza el carácter "?" para indicar los parámetros de entrada.
 - Cada signo de interrogación se identifica con un número, siendo 1 el primer símbolo.
 - En los siguientes pasos, se utilizan los métodos setXXX() para asignar un valor a estos parámetros.
 - Reciben dos parámetros: la posición del signo de interrogación que se desea parametrizar (empezando por 1) y el valor.
 - Existe un método setXXX() para cada tipo de dato de Java: setInt(), setFloat(), setString()...

La clase PreparedStatement

```
String sql = "SELECT * FROM Autos WHERE matricula = ?";  
PreparedStatement consulta = conn.prepareStatement(sql);  
consulta.setString(1, "2222YYY");  
ResultSet resultado = consulta.executeQuery();  
resultado.next();  
System.out.println("Precio del auto: " + resultado.getString("Precio"));
```

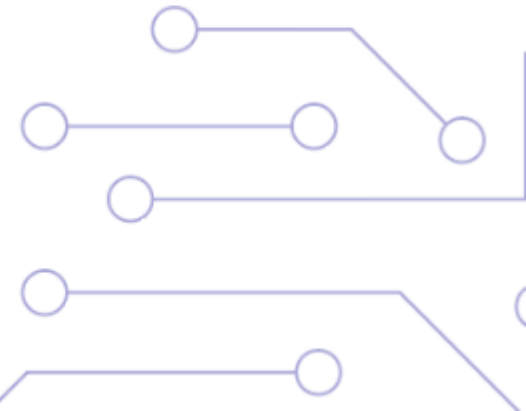
En este caso, el número de registro es un valor fijo, pero el valor también puede recuperarse de un objeto o de un cuadro de texto.

La clase PreparedStatement

```
String[] matriculas = new String[]{"1111XXX", "3333ZZZ"};
String sql = "SELECT * FROM Autos WHERE matricula = ?";
PreparedStatement consulta = conn.prepareStatement(sql);
for (String mat: matriculas)
{
    consulta.setString(1, mat);
    ResultSet resultado = consulta.executeQuery();
    resultado.next();
    System.out.println("Mat: "+mat+" - Precio: "+resultado.getString("Precio"));
}
```

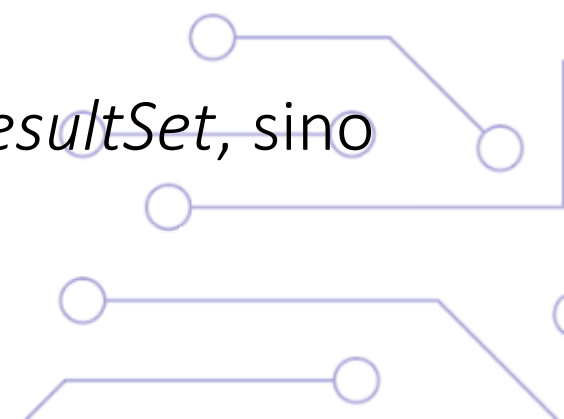
La clase PreparedStatement

```
String sql="INSERT INTO Propietarios(DNI, Nombre, Edad) VALORES(?, ?, ?)";  
PreparedStatement ps = conn.prepareStatement(sql);  
ps.setString(1, "888888888H");  
ps.setString(2, "Aitor");  
ps.setInt(3, 32);  
int n = ps.executeUpdate();  
System.out.println(n + " filas actualizadas.");
```



La clase CallableStatement

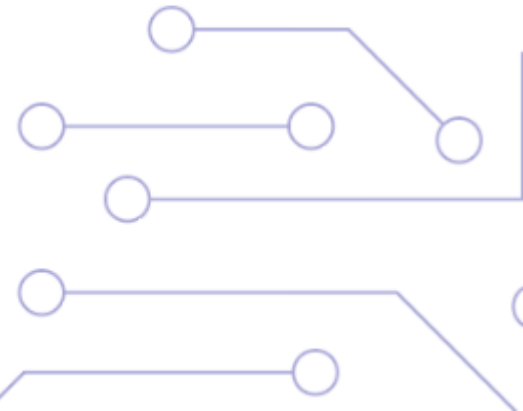
- Permite llamar a un procedimiento almacenado en la base de datos.
- El método *prepareCall()* se utiliza para preparar la consulta y siempre tiene la misma estructura:
 - "{call nombreprocedimiento (entrada-salida parametros como interrogacion)}"
 - Los métodos setXXX() se utilizan para asignar valores a los signos de interrogación.
- Si hay un parámetro de salida, debe registrarse, indicando su nombre y tipo.
- El método *execute()* se utiliza para ejecutar.
- ¡El parámetro de salida no se recibe de un objeto de tipo *ResultSet*, sino de la consulta!



La clase CallableStatement

```
PROCEDURE 'autodb'.bbPrecio'(OUT bb INTEGER)
BEGIN
SELECT avg(precio) INTO bb FROM autos;
END
```

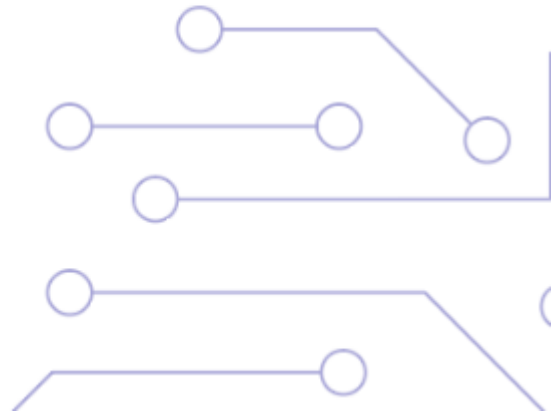
```
CallableStatement cs = conn.prepareCall("{call bbPrecio(?)}");
cs.registerOutParameter("bb" , Types.INTEGER);
cs.execute();
System.out.println(cs.getInt("bb"));
```



La clase CallableStatement

```
PROCEDURE 'autodb'.recogerModeloAuto' (IN NombrePropietario VARCHAR(255), OUT ModeloAuto VARCHAR(255))  
BEGIN  
SELECT Modelo INTO ModeloAuto  
FROM autos  
INNER JOIN propietarios ON autos.DNI = propietarios.DNI  
WHERE propietarios.Nombre = propietarioNombre;  
END
```

```
CallableStatement cs = conn.prepareCall("{call recogemodelodeAuto(?,?)}");  
cs.setString(1, "Leire");  
cs.registerOutParameter("modeloAuto" , Types.VARCHAR);  
cs.execute();  
System.out.println(cs.getString("modeloAuto"));
```



La clase CallableStatement

```
CallableStatement cs = conn.prepareCall("{call recogerModeloAuto(?,?)}");
String sql="SELECT Nombre FROM Propietarios";
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);
while (rs.next())
{
    String izena = rs.getString("Nombre");
    cs.setString(1, nombre);
    cs.registerOutParameter("modeloAuto", Types.VARCHAR);
    cs.execute();
    System.out.println(nombre + " - " + cs.getString("modeloAuto"));
}
```

